

spinalSynth: Filter Package Verification & Testing Plan

This document outlines the testing strategy, unit test coverage, and integration test setup for the State Variable Filter (SVF) module, aligning with the conventions established in the `envelope` and `oscillator` test suites.

1. Test Architecture Overview

All tests are implemented using **ScalaTest** (`AnyFunSuite`) and **SpinalSim**. They reside in the test package `synth.filter` under the directory: `src/test/scala/synth/filter/`

The testing is split into two levels:

1. **Unit Tests:** Verifying individual submodules (`ParameterMapper`, `FilterCore`, `FilterMux`) in isolation.
 2. **Integration Tests:** Verifying the top-level wrapper (SVF) including state machine sequencing, enable/disable behavior, reset behavior, next-phaseTick flow synchronization, and filter functionality.
-

2. Unit Tests

2.1 ParameterMapperSim (`ParameterMapperSim.scala`)

Verifies lookup ROM generation and correctness of exponential and quadratic mappings.

- **Test cases:**
 - **Exponential Cutoff Mapping:**
 - Verify index 0 maps to minimum coeff 10.
 - Verify index 255 maps to maximum coeff 4095.
 - Verify monotonic growth across intermediate values (e.g. 64, 128, 192) and compliance with the mapping equation.
 - **Quadratic Resonance Mapping:**
 - Verify index 0 maps to maximum coefficient 255 (representing minimum resonance/maximum damping).
 - Verify index 255 maps to minimum coefficient 4 (representing maximum resonance/minimum damping).
 - Verify quadratic decay across intermediate values (e.g. 64, 128, 192).

2.2 FilterCoreSim (`FilterCoreSim.scala`)

Verifies the time-multiplexed arithmetic FSM sequencing, shared operator scheduling, and register clearing.

- **Test cases:**
 - **FSM Sequencing & Timing:**
 - Assert that on `phaseTick`, the FSM transitions from `IDLE` through the 8 execution states and back to `IDLE`.
 - Verify that the `done` flag pulses High exactly 8 cycles after `phaseTick`.
 - **ClockDomain Reset Behavior:**
 - Assert that during global system reset assertion, all internal registers (including `lp`, `bp`, and FSM state registers) are properly initialized to 0 / `IDLE`.
 - Verify that the FSM recovers cleanly and stays in `IDLE` after reset release until the next `phaseTick`.
 - **State Reset (Clear):**
 - Assert that when `clear` is asserted (independent of global reset), internal state registers (`lp` and `bp`) are immediately set to 0 and the FSM remains in `IDLE`.
 - **Arithmetic Execution & Normal Vectors:**

- Drive known values (e.g. `sampleIn = 4000`, `lp = 0`, `bp = 0`, coefficients) and step the simulator cycle-by-cycle.
- Compare internal register states (`hp`, `bp`, `lp`, `resTerm`, etc.) against hand-calculated values for each state transition to verify shared multiplier and adder multiplexer routing.
- **Extreme Values and Overflow/Wrap-Around Verification:**
 - **State-Overflow Test Vector:** Drive maximum positive `sampleIn = 32767` and force state registers to maximum out-of-phase values (e.g. `lp = -8388608` or `bp = -8388608`) with extreme coefficients to intentionally saturate internal intermediate additions (e.g. `tempSub = inputExt - lp = 32767 - (-8388608) = 8421375`, which exceeds the signed 24-bit range of 8388607).
 - **Wrap-around Verification:** Confirm that SpinalHDL's `.resize(24 bits)` truncation wraps around predictably (e.g., verifying that 8421375 wraps to -8354433), ensuring the hardware matches our software bit-accurate model under severe feedback/clipping conditions.
 - **Negative Limit Multiplier Stability:** Drive `bp = -8388608` and `resonanceCoeff = 255`. Verify that `resTerm` is correctly shifted and sign-extended without multiplier sign-bit corruption.

2.3 FilterMuxSim (FilterMuxSim.scala)

Verifies output response selection and fixed-point formatting.

- **Test cases:**
 - **Mode Selection Muxing:**
 - Force distinct test values on `lp`, `bp`, and `hp` ports.
 - Sweep mode through 00 (LP), 01 (BP), and 10 (HP), and verify the output payload matches the selected port scaled down to 16 bits.
 - **Saturating Resize:**
 - Drive known internal values that exceed the signed 16-bit range (e.g. `0x123456` or `-0x154321`) and verify that the output saturates correctly to 32767 or -32768 instead of performing MSB truncation.

3. Integration Tests

3.1 SVFSim (SVFSim.scala)

Verifies the full pipeline assembly, register control path, next-phaseTick flow synchronization, and functional filtering responses.

- **Test cases:**
 - **Integration Reset Behavior:**
 - Assert that during active startup reset, `sampleOut.valid` is False and `sampleOut.payload` is 0.
 - Assert that outputs remain quiet after reset release before ticks start.
 - **Saturation under Overshoot:**
 - Feed a full-scale step input (+32767) continuously to the lowpass filter mode.
 - Verify that the filter output peaks and stabilizes at 32767 and does not wrap around to a negative number at any point.
 - **Next-phaseTick Output Flow Synchronization (Critical):**
 - Apply `sampleIn.valid` (representing the incoming sample tick).
 - Wait for the FSM to finish calculation.
 - Assert that the output flow `sampleOut.valid` does **not** assert early.
 - Assert that `sampleOut.valid` is asserted **exactly** on the next `phaseTick` boundary (exactly 50 clock cycles after the input sample was presented).
 - **Enable / Disable Behavior:**
 - **Enabled:** Apply input flow and verify the output changes according to filter responses.
 - **Disabled:** Deassert `enable`. Verify that `sampleOut.payload` is driven to 0 immediately, internal state registers `lp/bp` reset to 0, and the `sampleOut.valid` signal continues to pulse in sync with `phaseTick` to keep downstream modules active.
 - **Filter Frequency Response (Monotonicity & Behavior):**

- **Lowpass Mode:** Feed a DC offset (low frequency) and a high-frequency toggle (Nyquist). Verify the low frequency passes with minimal attenuation while the high-frequency toggle is strongly attenuated.
- **Highpass Mode:** Feed a DC offset and a high-frequency toggle. Verify the DC offset is blocked (output = 0) and the high frequency passes.
- **Bandpass Mode:** Feed a frequency sweep and verify the resonant frequency peaks at the configured cutoff frequency.